

Software Testing

Outline

- Overview of Software Testing
- Black Box / Functional Testing
- White Box / Structural Testing
- Integration Testing
- System Testing

Overview of Software Testing

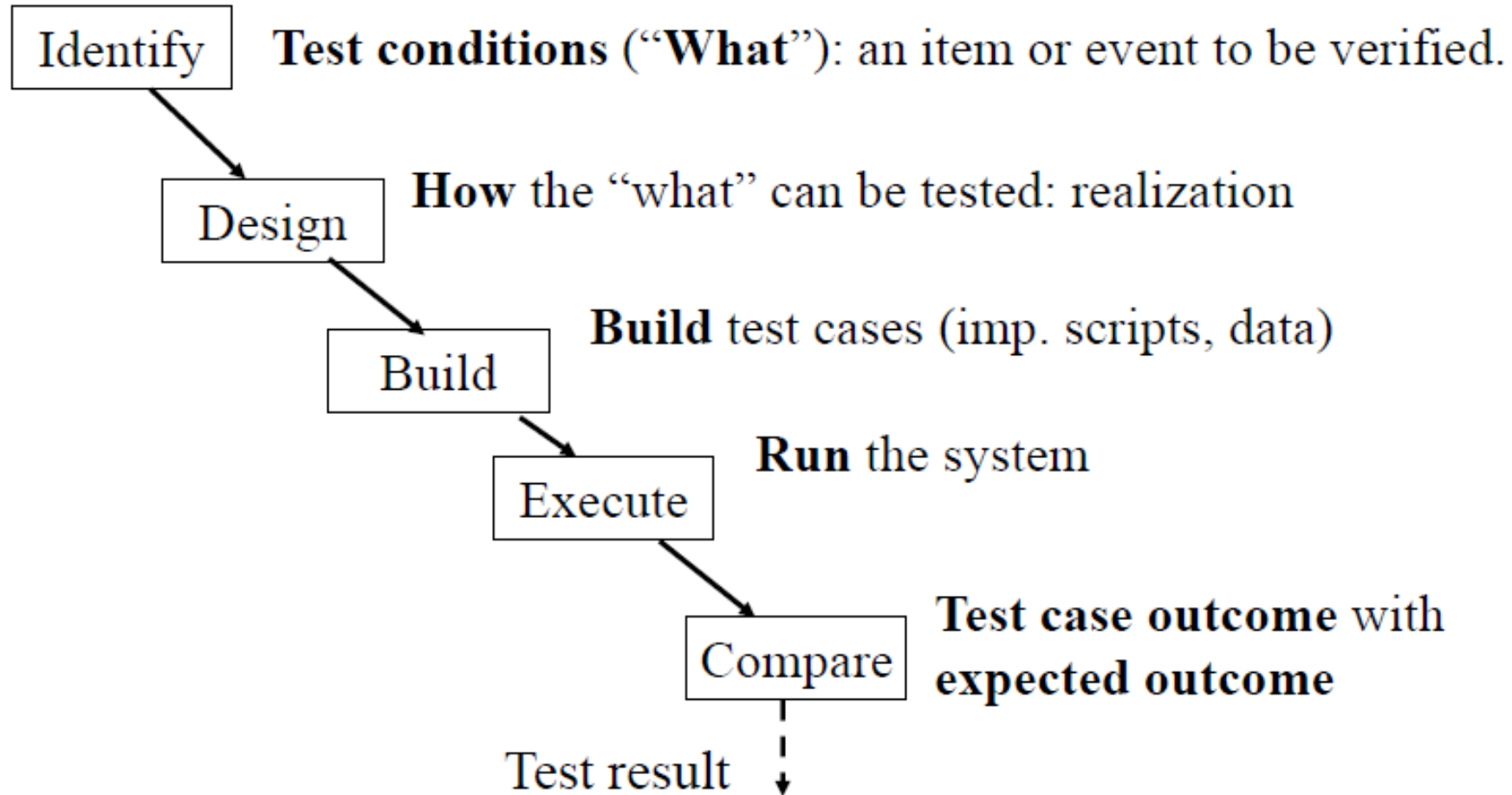
Software Testing: Definition

- Software testing is the **process** of evaluating and verifying that a software application or system **meets its requirements and functions as expected** <https://www.ibm.com/think/topics/software-testing>
- Testing can never establish the correctness of computer software. It can only find defects, not prove that there are none.”

Why Testing?

- To discover more problems
 - Meyer's definition: "Software testing is a process of finding the errors in the product"
 - Testing never ends
- To ensure quality
 - Check whether the product matches with the requirements

Testing Activities



Test Stopping Criteria

- Meet deadline, exhaust budget, ... (management decision)
- Achieved desired coverage
- Achieved desired level of failure intensity

Black Box / Functional Testing

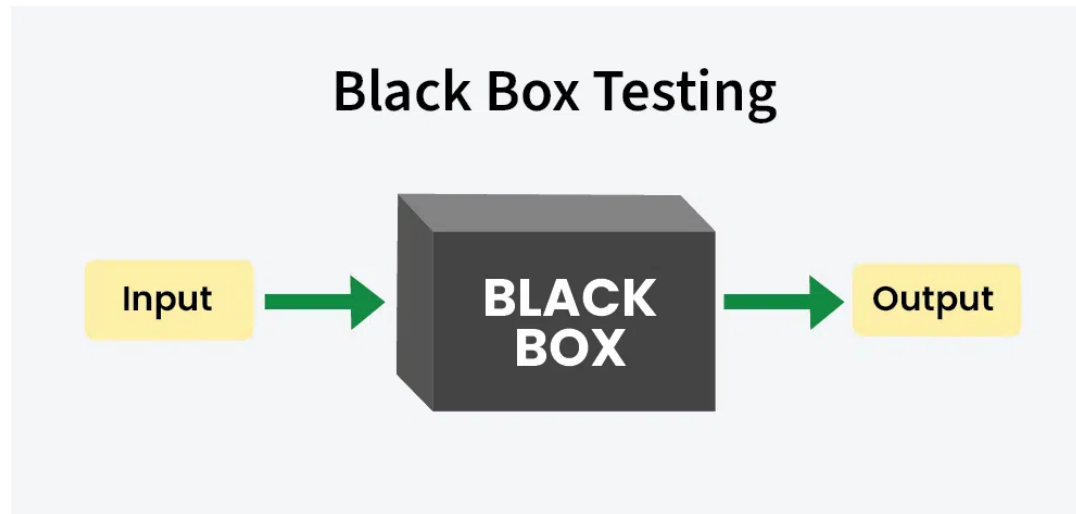


Black-box Testing

- The only information used is the specification of the software.
- Functional test cases have two distinct advantages :
 - They are independent of how the software is developed, so if the implementation changes, the test cases are still useful;
 - Test case development can occur in parallel with the implementation, thereby reducing overall project development interval;
- Functional test cases suffer from two problems :
 - Significant redundancies may exist among test cases;
 - Compounded by the possibility of gaps of untested software;

Definition of Black-box Testing

- **Black-box testing** is a software testing methodology where the system under test is evaluated based on its **external behavior, without knowledge** of its internal code, structure, or implementation details. Testers focus on **inputs and outputs**, validating whether the software functions according to specified requirements, user expectations, and desired outcomes.



Black-box Testing

- Focus: I/O behavior. If for any given input, we can predict the output, then the module passes the test.
 - Almost always impossible to generate all possible inputs ("test cases")
- Goal: Reduce number of test cases by equivalence partitioning:
 - Divide input conditions into equivalence classes
 - Choose test cases for each equivalence class. (Example: If an object is supposed to accept a negative number, testing one negative number is enough)

Black-box Testing

- **Boundary Value Testing (BVT)**

- Normal boundary value testing
- Robust boundary value testing
- Worst-case boundary value testing
- Robust worst-case boundary value testing

- **Equivalence Class Testing**

- Weak Normal Equivalence Class Testing
- Strong Normal Equivalence Class Testing
- Weak Robust Equivalence Class Testing
- Strong Robust Equivalence Class Testing

- **Decision Table Based Testing**

Boundary Value Testing : Overview

- Any program can be considered to be a function
 - Program inputs form its **domain**
 - Program outputs form its **range**
- Boundary value analysis is the best known functional testing technique.
- The objective of functional testing is to use knowledge of the functional nature of a program to identify test cases.
- Historically, functional testing has focused on the input domain, but it is a good supplement to consider test cases based on the range **(outputs)** as well.



Boundary Value Testing: Overview

- Boundary value testing focuses on the boundary of the input space to identify test cases.
- The rationale behind boundary value analysis is that **errors tend to occur near the extreme values of an input variable.**
- Programs written in not strongly typed languages are more appropriate candidates for boundary value testing

Boundary Value Testing: Overview

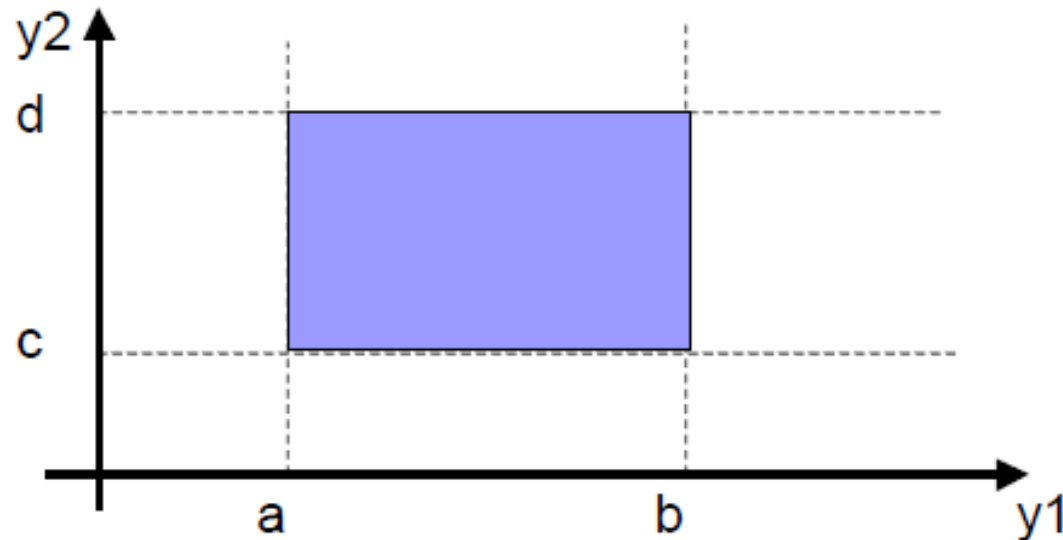
- BVT tends to generate **more test cases** with **poorer test coverage** than domain or equivalence testing.
- Due to its simplicity, BVT test case generation can be easily automated.
- In our discussion we will assume a program P accepting two inputs y_1 and y_2 such that $a \leq y_1 \leq b$ and $c \leq y_2 \leq d$

Valid Input for Program P

- consider the following function:

$$f(y_1, y_2), \text{ where } a \leq y_1 \leq b, c \leq y_2 \leq d$$

- boundary inequalities of **n** input variables define an **n**-dimensional input space (domain):



Variant of Boundary Value Testing

- Two independent considerations: -
 - Concerned with invalid values of variables?
 - Only valid values – Normal
 - Both valid and invalid values – Robust
 - Single fault assumption?
 - Fault are due to invalidity of single variable.
 - Fault are due to the interaction among two or more variables.
- Two considerations yield 4 type of boundary value testing: -
 - Normal boundary value testing
 - Robust boundary value testing
 - Worst-case boundary value testing
 - Robust worse-case value testing

Normal Boundary Value Testing

- The basic idea in boundary value analysis is to select input variable values at their:
 - Minimum
 - Just above the minimum
 - A nominal value ($(\text{min} + \text{max})/2$)
 - Just below the maximum
 - Maximum
- We use the conventions (min, min+, nom, max-, and max) to refer to these values



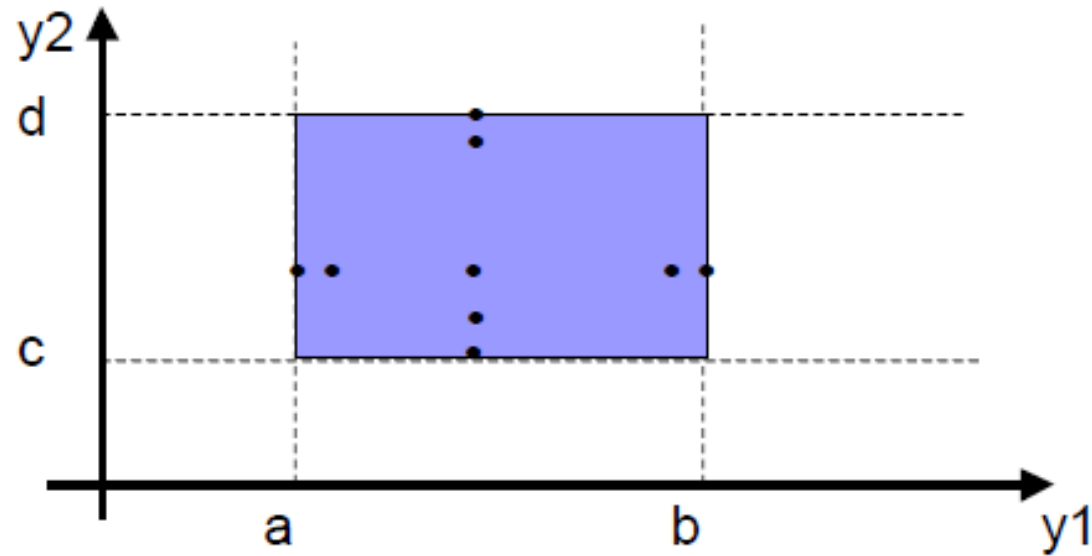
Normal Boundary Value Testing

- Augmented by the **single fault assumption** principle.

“Failures occur rarely as the result of the simultaneous occurrence of two (or more) faults”

- In this respect, normal boundary value test cases can be obtained by holding all but one at the nominal values and let the remaining variable assume the min, min+, nom, max–, and max values, repeating this for each variable. Thus, for a function of n variables, boundary value analysis yields $4n + 1$ unique test cases.

Normal Boundary Value Testing for Program P



$$T = \{ \langle y_{1nom}, y_{2min} \rangle, \langle y_{1nom}, y_{2min+} \rangle, \langle y_{1nom}, y_{2nom} \rangle, \\ \langle y_{1nom}, y_{2max-} \rangle, \langle y_{1nom}, y_{2max+} \rangle, \langle y_{1min}, y_{2nom} \rangle, \\ \langle y_{1min+}, y_{2nom} \rangle, \langle y_{1max-}, y_{2nom} \rangle, \langle y_{1max}, y_{2nom} \rangle \}$$

Generalizing Boundary Value Analysis

- Boundary value analysis is a test design technique used to identify and select input or output values.
- The basic boundary value analysis for normal boundary value testing can be generalized in two ways:
 - By the number of variables - $(4n + 1)$ test cases for n variables
 - By the kinds of ranges of variables
 - Bounded discrete variable – 5 test values (min, min+, nom, max-, max)
 - Unbounded discrete (no upper or lower bounds clearly defined), create “artificial” bounds
 - Logical variables present a problem for boundary value analysis, only true and false

Limitations of Boundary Value Analysis

- Boundary value analysis works well when the program to be tested is a function of several independent variables that represent bounded physical quantities.
- Boundary value analysis selected test data with no consideration of the function of the program, nor of the semantic meaning of the variables.
- We shall distinguish between physical and logical type of variables (e.g. temperature, pressure speed, or PIN numbers, telephone numbers etc.) . BVT works well for physical variables but not for logical variables.

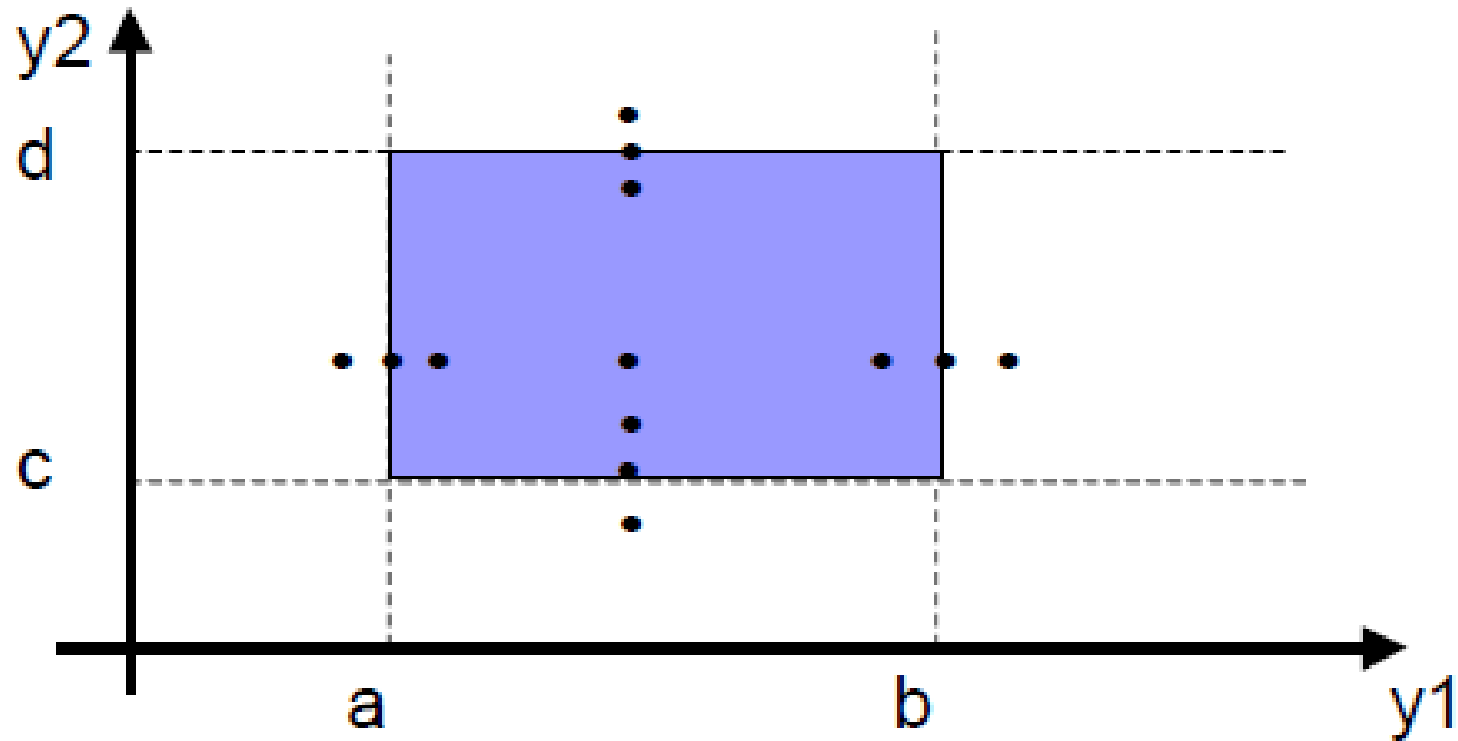
Robust Boundary Value Testing

- Robustness testing is a simple extension of boundary value analysis.
- In addition to the five boundary value analysis values of variables, we add values slightly greater than the maximum (max+) and a value slightly less than the minimum (min-).
- In this respect, for a function of n variables, there will be $6n + 1$ unique test cases.
- The main value of robustness testing is to force attention on exception handling.

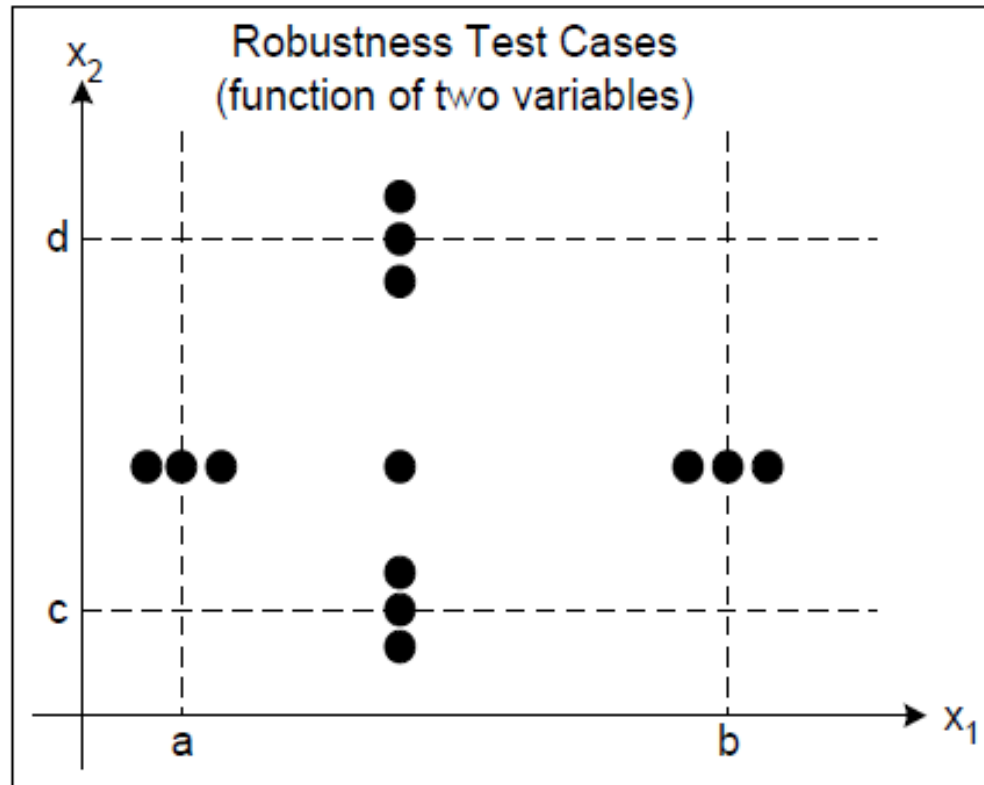
Robust Boundary Value Testing

- In some strongly typed languages values beyond the predefined range will cause a run-time error.
- It is a choice of using a weak typed language with exception handling or a strongly typed language with explicit logic to handle out of range values.

Robustness Test Cases for Program P



Robustness Test Cases for Program P

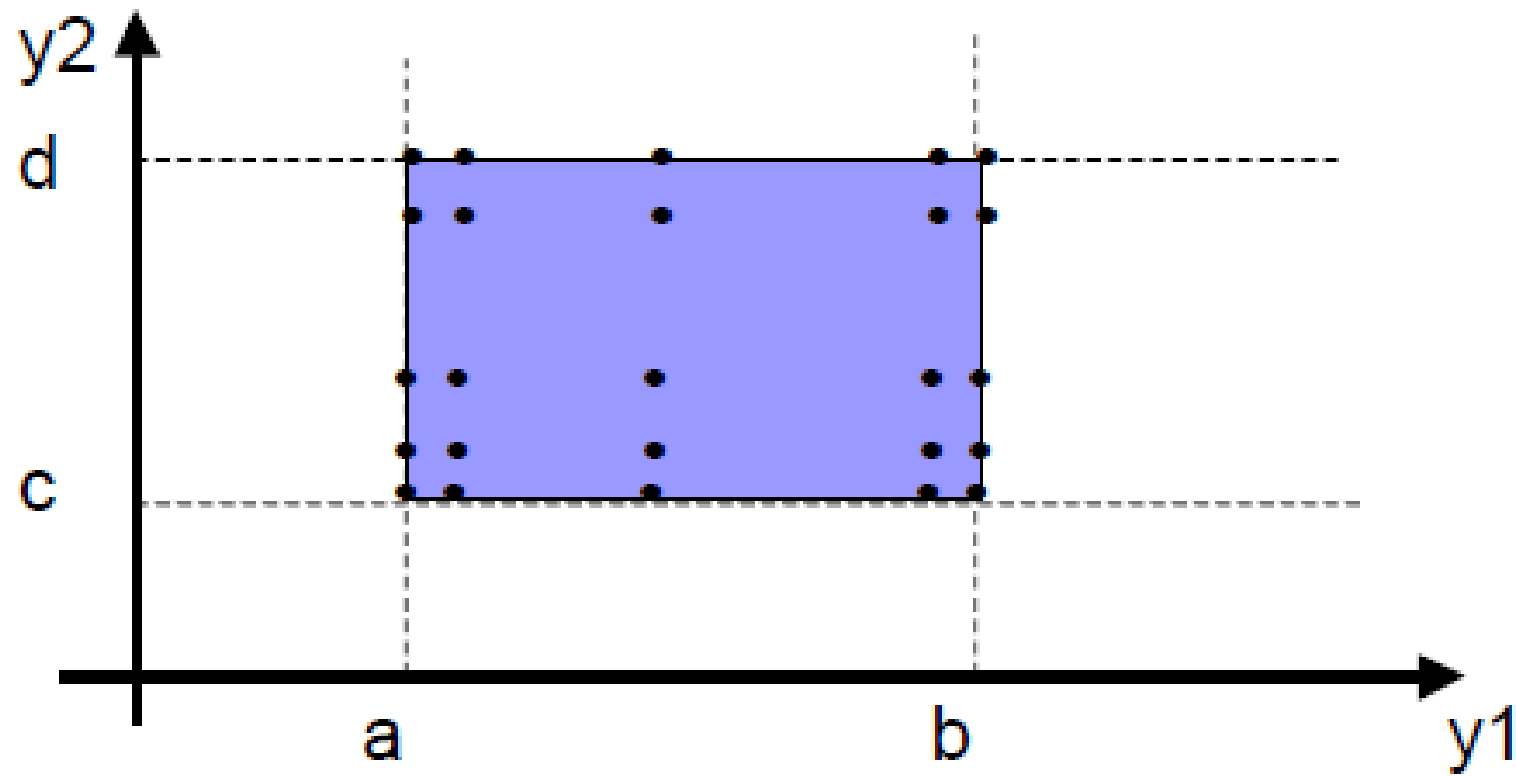


$\langle x_{1nom}, x_{2min} \rangle$
 $\langle x_{1nom}, x_{2min+} \rangle$
 $\langle x_{1nom}, x_{2nom} \rangle$
 $\langle x_{1nom}, x_{2max-} \rangle$
 $\langle x_{1nom}, x_{2max} \rangle$
 $\langle x_{1min}, x_{2nom} \rangle$
 $\langle x_{1min+}, x_{2nom} \rangle$
 $\langle x_{1nom}, x_{2nom} \rangle$
 $\langle x_{1max-}, x_{2nom} \rangle$
 $\langle x_{1max}, x_{2nom} \rangle$
 $\langle x_{1min-}, x_{2nom} \rangle$
 $\langle x_{1nom}, x_{2max+} \rangle$
 $\langle x_{1max+}, x_{2nom} \rangle$
 $\langle x_{1nom}, x_{2min-} \rangle$

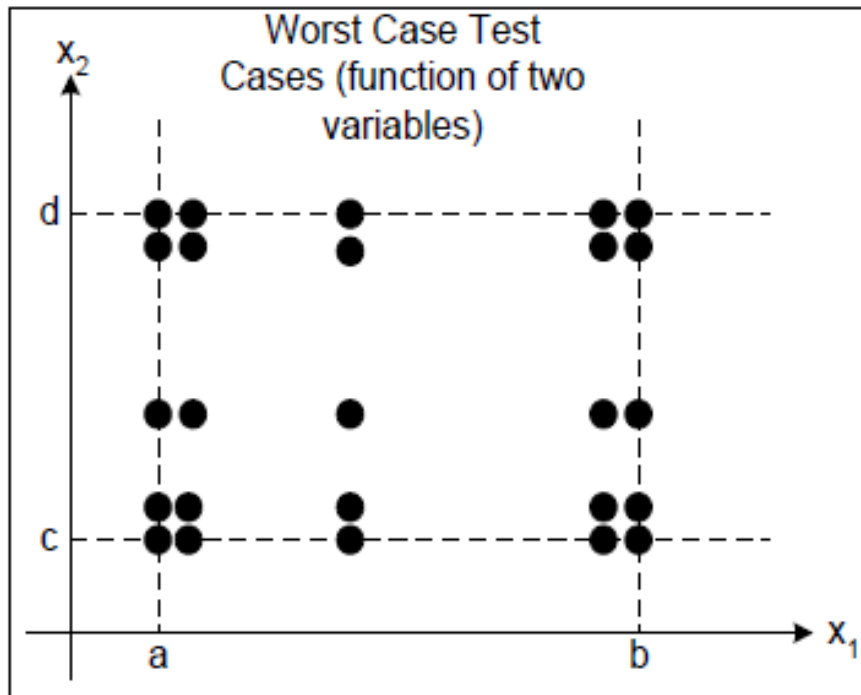
Worst-case Boundary Value Testing

- In worst case testing we reject the single fault assumption and we are interested in what happens when more than one variable has an extreme value.
- Considering that we have five different values that can be considered during normal boundary value testing for one variable, now we take the Cartesian product of these possible values for 2, 3, ... n variables. In this respect we can have 5^n test cases for n input variables.
- The best application of worst case testing is where physical variables have numerous interactions and failure of a program is costly.

Worst-case Boundary Value Testing for Program P



Worst-case Boundary Value Testing for Program P



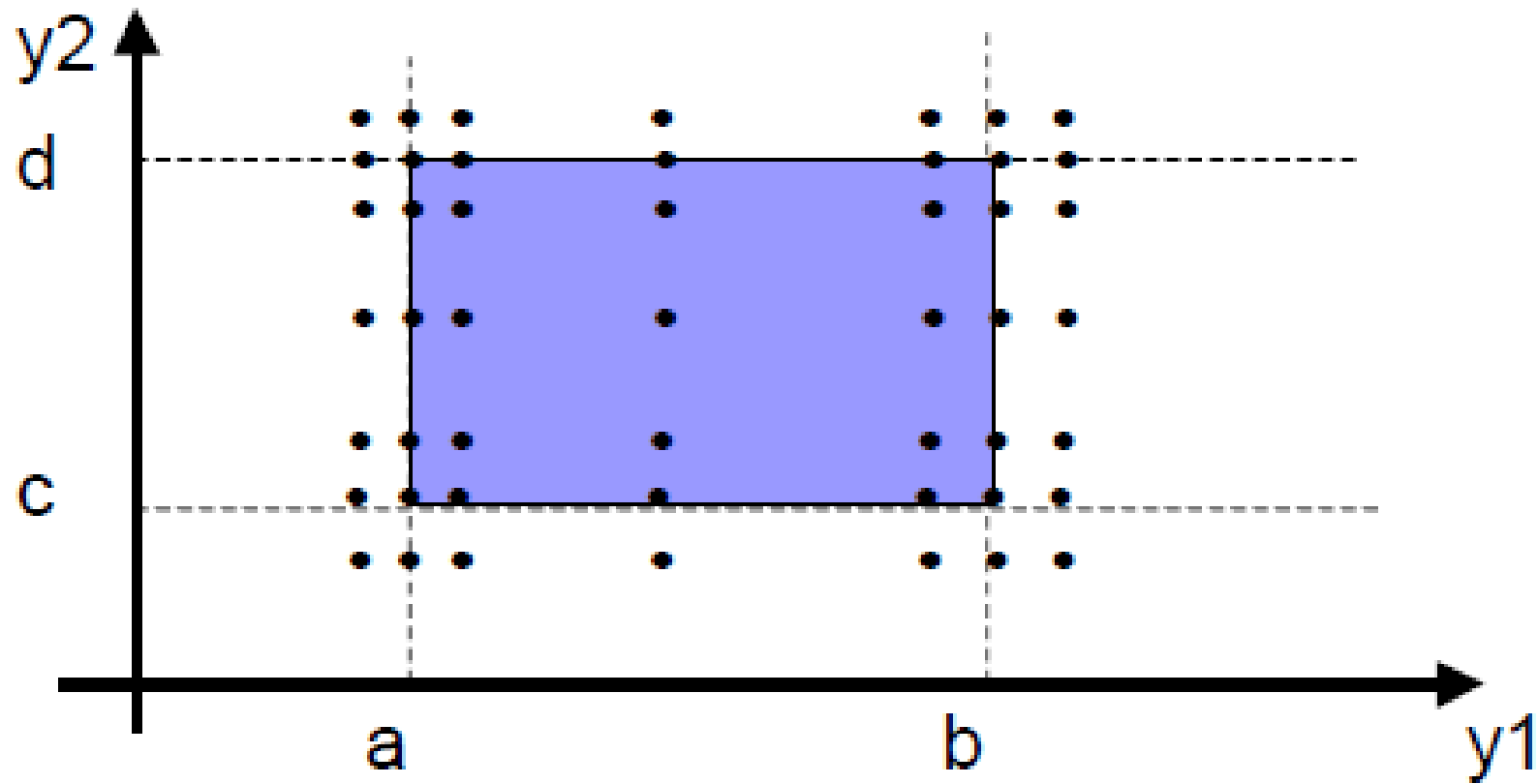
$\langle X_{1min}, X_{2min} \rangle$
 $\langle X_{1min}, X_{2min+} \rangle$
 $\langle X_{1min}, X_{2nom} \rangle$
 $\langle X_{1min}, X_{2max-} \rangle$
 $\langle X_{1min}, X_{2max} \rangle$
 $\langle X_{1min+}, X_{2min} \rangle$
 $\langle X_{1min+}, X_{2min+} \rangle$
 $\langle X_{1min+}, X_{2nom} \rangle$
 $\langle X_{1min+}, X_{2max-} \rangle$
 $\langle X_{1min+}, X_{2max} \rangle$
 $\langle X_{1nom}, X_{2min} \rangle$
 $\langle X_{1nom}, X_{2min+} \rangle$
 $\langle X_{1nom}, X_{2nom} \rangle$
 $\langle X_{1nom}, X_{2max-} \rangle$
 $\langle X_{1nom}, X_{2max} \rangle$

$\langle X_{1max-}, X_{2min} \rangle$
 $\langle X_{1max-}, X_{2min+} \rangle$
 $\langle X_{1max-}, X_{2nom} \rangle$
 $\langle X_{1max-}, X_{2max-} \rangle$
 $\langle X_{1max-}, X_{2max} \rangle$
 $\langle X_{1max}, X_{2min} \rangle$
 $\langle X_{1max}, X_{2min+} \rangle$
 $\langle X_{1max}, X_{2nom} \rangle$
 $\langle X_{1max}, X_{2max-} \rangle$
 $\langle X_{1max}, X_{2max} \rangle$

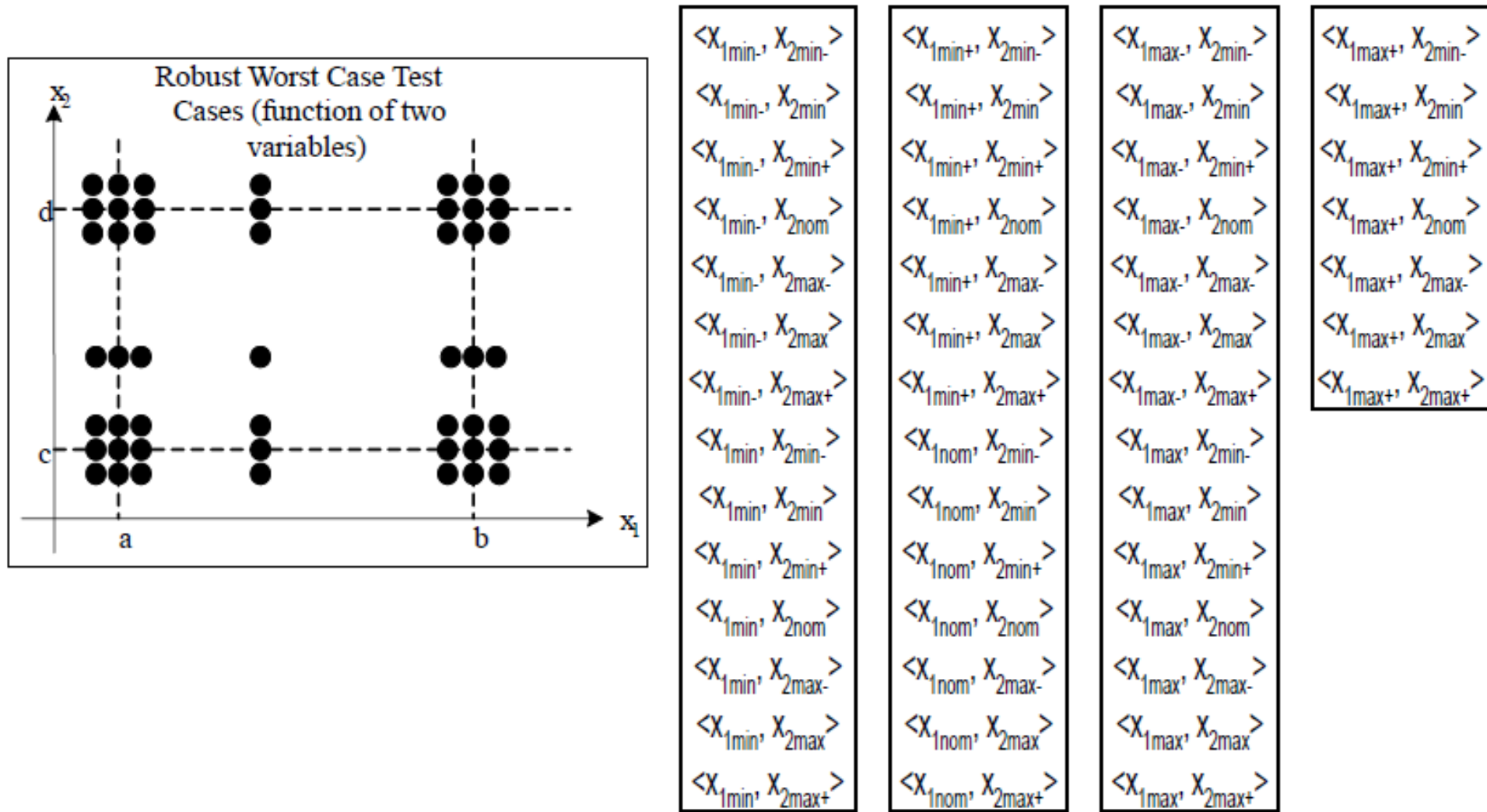
Robust Worst-case Boundary Value Testing

- Worst case testing can be further augmented by considering robust worst case testing (i.e. adding slightly out of bounds values to the five already considered). This involves the Cartesian product of the seven-element sets we used in robustness testing.
- In this respect we can have 7^n test cases for n input variables.

Robust Worst-case Boundary Value Testing for Program P



Robust Worst-case Boundary Value Testing for Program P



Example – The Triangle Problem

- Problem Statement
 - Input: 3 integers (sides of a triangle)
 - Output: Type of Triangle (Equilateral, Isosceles, Scalene or NotATriangle)
 - Extended Version: Additional Output Type: Right Triangle

Example – The Triangle Problem

- In the problem statement, no conditions are specified on the triangle sides, other than being integers.
- Obviously, the lower bounds of the ranges are all 1.
- We arbitrarily take 200 as an upper bound.
- For each side, the test values are {1, 2, 100, 199, 200}.
- Robust boundary value test cases will add {0, 201}.
- The table on the next page contains Normal Boundary value test cases using these ranges.
 - Test cases 3, 8, and 13 are identical – Redundant
 - There is no test case for scalene triangles

Example

Table 5.1 Normal Boundary Value Test Cases

Case	a	b	c	<i>Expected Output</i>
1	100	100	1	Isosceles
2	100	100	2	Isosceles
3	100	100	100	Equilateral
4	100	100	199	Isosceles
5	100	100	200	Not a triangle
6	100	1	100	Isosceles
7	100	2	100	Isosceles
8	100	100	100	Equilateral
9	100	199	100	Isosceles
10	100	200	100	Not a triangle
11	1	100	100	Isosceles
12	2	100	100	Isosceles
13	100	100	100	Equilateral
14	199	100	100	Isosceles
15	200	100	100	Not a triangle

Example – The NextDate Function

- The function takes 3 variables, month, the day, and the year. It return the next day of the input.
- We could encode these, so that January would correspond to 1, February to 2, and so on.
- In this example, we use Worst-case boundary value testing. All 125 worst-case test cases for NextDate are listed in the table on next page.
- Examine the following: -
 - Gaps of untested functionality
 - Redundant testing
- Questions: -
 - Would anyone actually want to test January 1 in five different years?
 - Is the end of February tested sufficiently?

Case	Month	Day	Year	Expected Output
1	1	1	1812	1, 2, 1812
2	1	1	1813	1, 2, 1813
3	1	1	1912	1, 2, 1912
4	1	1	2011	1, 2, 2011
5	1	1	2012	1, 2, 2012
6	1	2	1812	1, 3, 1812
7	1	2	1813	1, 3, 1813
8	1	2	1912	1, 3, 1912
9	1	2	2011	1, 3, 2011
10	1	2	2012	1, 3, 2012
11	1	15	1812	1, 16, 1812
12	1	15	1813	1, 16, 1813
13	1	15	1912	1, 16, 1912
14	1	15	2011	1, 16, 2011
15	1	15	2012	1, 16, 2012
16	1	30	1812	1, 31, 1812
17	1	30	1813	1, 31, 1813
18	1	30	1912	1, 31, 1912
19	1	30	2011	1, 31, 2011
20	1	30	2012	1, 31, 2012
21	1	31	1812	2, 1, 1812
22	1	31	1813	2, 1, 1813
23	1	31	1912	2, 1, 1912
24	1	31	2011	2, 1, 2011
25	1	31	2012	2, 1, 2012
26	2	1	1812	2, 2, 1812
27	2	1	1813	2, 2, 1813
28	2	1	1912	2, 2, 1912

Case	Month	Day	Year	Expected Output
29	2	1	2011	2, 2, 2011
30	2	1	2012	2, 2, 2012
31	2	2	1812	2, 3, 1812
32	2	2	1813	2, 3, 1813
33	2	2	1912	2, 3, 1912
34	2	2	2011	2, 3, 2011
35	2	2	2012	2, 3, 2012
36	2	15	1812	2, 16, 1812
37	2	15	1813	2, 16, 1813
38	2	15	1912	2, 16, 1912
39	2	15	2011	2, 16, 2011
40	2	15	2012	2, 16, 2012
41	2	30	1812	Invalid date
42	2	30	1813	Invalid date
43	2	30	1912	Invalid date
44	2	30	2011	Invalid date
45	2	30	2012	Invalid date
46	2	31	1812	Invalid date
47	2	31	1813	Invalid date
48	2	31	1912	Invalid date
49	2	31	2011	Invalid date
50	2	31	2012	Invalid date
51	6	1	1812	6, 2, 1812
52	6	1	1813	6, 2, 1813
53	6	1	1912	6, 2, 1912
54	6	1	2011	6, 2, 2011
55	6	1	2012	6, 2, 2012
56	6	2	1812	6, 3, 1812
57	6	2	1813	6, 3, 1813

Case	Month	Day	Year	Expected Output
58	6	2	1912	6, 3, 1912
59	6	2	2011	6, 3, 2011
60	6	2	2012	6, 3, 2012
61	6	15	1812	6, 16, 1812
62	6	15	1813	6, 16, 1813
63	6	15	1912	6, 16, 1912
64	6	15	2011	6, 16, 2011
65	6	15	2012	6, 16, 2012
66	6	30	1812	7, 1, 1812
67	6	30	1813	7, 1, 1813
68	6	30	1912	7, 1, 1912
69	6	30	2011	7, 1, 2011
70	6	30	2012	7, 1, 2012
71	6	31	1812	Invalid date
72	6	31	1813	Invalid date
73	6	31	1912	Invalid date
74	6	31	2011	Invalid date
75	6	31	2012	Invalid date
76	11	1	1812	11, 2, 1812
77	11	1	1813	11, 2, 1813
78	11	1	1912	11, 2, 1912
79	11	1	2011	11, 2, 2011
80	11	1	2012	11, 2, 2012
81	11	2	1812	11, 3, 1812
82	11	2	1813	11, 3, 1813
83	11	2	1912	11, 3, 1912
84	11	2	2011	11, 3, 2011
85	11	2	2012	11, 3, 2012
86	11	15	1812	11, 16, 1812

Case	Month	Day	Year	Expected Output
87	11	15	1813	11, 16, 1813
88	11	15	1912	11, 16, 1912
89	11	15	2011	11, 16, 2011
90	11	15	2012	11, 16, 2012
91	11	30	1812	12, 1, 1812
92	11	30	1813	12, 1, 1813
93	11	30	1912	12, 1, 1912
94	11	30	2011	12, 1, 2011
95	11	30	2012	12, 1, 2012
96	11	31	1812	Invalid date
97	11	31	1813	Invalid date
98	11	31	1912	Invalid date
99	11	31	2011	Invalid date
100	11	31	2012	Invalid date
101	12	1	1812	12, 2, 1812
102	12	1	1813	12, 2, 1813
103	12	1	1912	12, 2, 1912
104	12	1	2011	12, 2, 2011
105	12	1	2012	12, 2, 2012
106	12	2	1812	12, 3, 1812
107	12	2	1813	12, 3, 1813
108	12	2	1912	12, 3, 1912
109	12	2	2011	12, 3, 2011
110	12	2	2012	12, 3, 2012
111	12	15	1812	12, 16, 1812
112	12	15	1813	12, 16, 1813
113	12	15	1912	12, 16, 1912
114	12	15	2011	12, 16, 2011
115	12	15	2012	12, 16, 2012

<i>Case</i>	<i>Month</i>	<i>Day</i>	<i>Year</i>	<i>Expected Output</i>
116	12	30	1812	12, 31, 1812
117	12	30	1813	12, 31, 1813
118	12	30	1912	12, 31, 1912
119	12	30	2011	12, 31, 2011
120	12	30	2012	12, 31, 2012
121	12	31	1812	1, 1, 1813
122	12	31	1813	1, 1, 1814
123	12	31	1912	1, 1, 1913
124	12	31	2011	1, 1, 2012
125	12	31	2012	1, 1, 2013

Black-box Testing

- Boundary Value Testing (BVT)
 - Normal boundary value testing
 - Robust boundary value testing
 - Worst-case boundary value testing
 - Robust worst-case boundary value testing
- **Equivalence Class Testing**
 - Weak Normal Equivalence Class Testing
 - Strong Normal Equivalence Class Testing
 - Weak Robust Equivalence Class Testing
 - Strong Robust Equivalence Class Testing
 - Decision Table Based Testing

Equivalence Class Testing - Overview

- BVT assumes that input variables are independent of one another
- BVT tends to generate more test cases with poorer test coverage (with the independence assumption) than domain or equivalence testing.
- Equivalence classes form a partition of a set that is a collection of mutually disjoint subsets whose union is the entire set.
- Two important implications for testing:
 - The fact that the entire set is represented provides a form of completeness
 - The disjointness assures a form of non-redundancy

Equivalence Classes

- The idea of equivalence class testing is to identify test cases by using one element from each equivalence class.
- If the equivalence classes are chosen wisely this greatly reduces the potential redundancy among test cases.
- The key point in equivalence class testing is the choice of the **equivalence relation that determines the classes** (partitions).

Equivalence Class Selection

- tends to much of a “craft”:
 - no dependence on knowledge of code, only the specification
 - needs knowledge of input domain that usually goes beyond what an interface design specification provides
 - must understand how inputs are mutually dependent

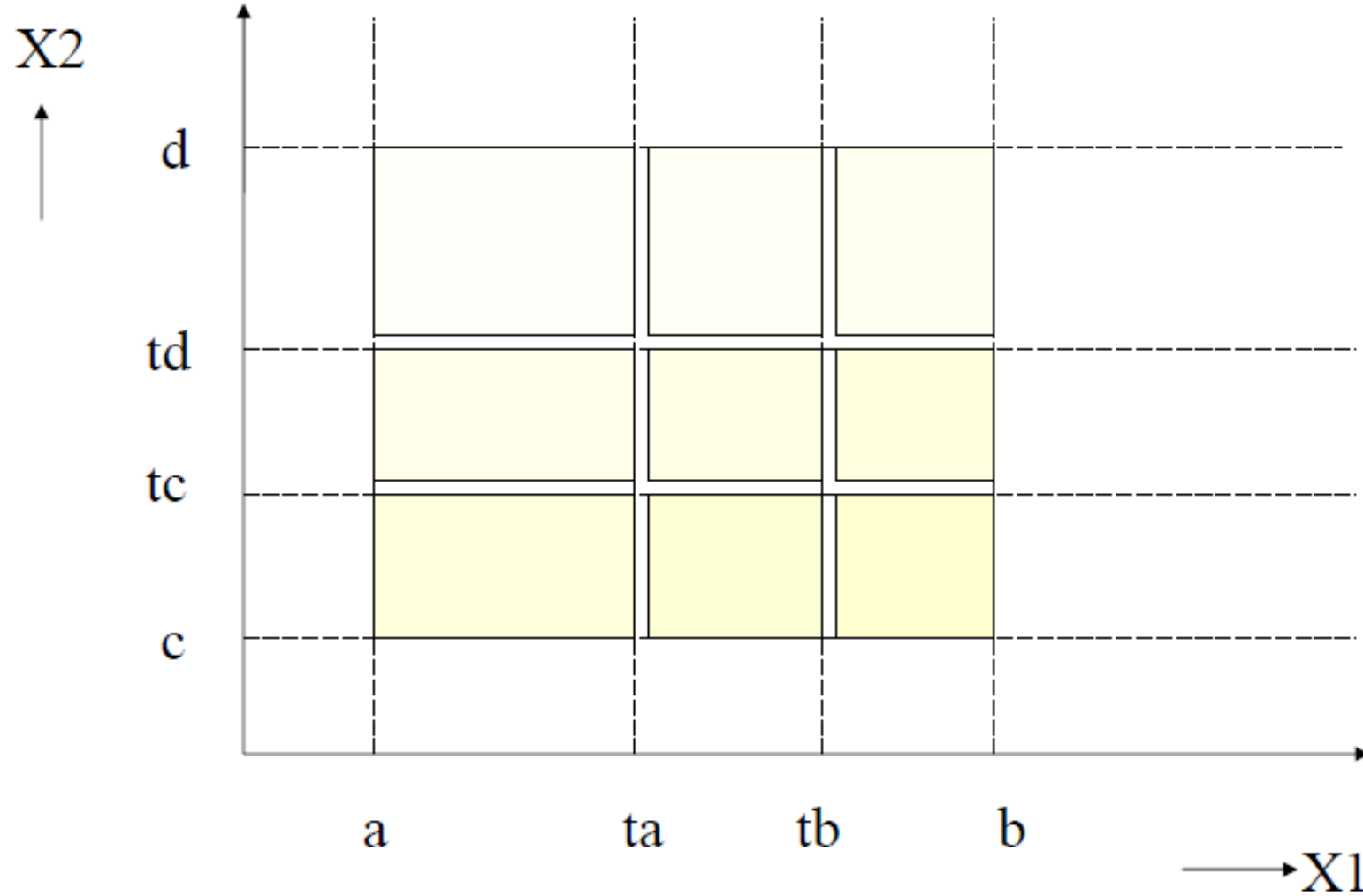
Equivalence Class Testing for 2-variable function

- Consider a function $f(x_1, x_2)$ where the values of x_1 and x_2 are defined to be

$$a \leq x_1 \leq b \quad \text{and} \quad c \leq x_2 \leq d$$

- Assume the following equivalence classes for x_1
 - $\{a, a+1, \dots, t_a\}, \{t_a+1, t_a+2, \dots, t_b\}, \{t_b+1, t_b+2, \dots, b\}$
- Assume the following equivalence classes for x_2
 - $\{c, c+1, \dots, t_c\}, \{t_c+1, t_c+2, \dots, t_d\}, \{t_d+1, t_d+2, \dots, d\}$

Graphical Representation



Test Data selection for 2-variable function

- Choose one value from each shaded region
 - Each value serves as a representative for all values in that region
- Total number of values = $n * m$ where
 - 'n' is the number of partitions for input X1
 - 'm' is the number of partitions for input X2

Equivalence Class Testing for 2-variable function – Robustness Testing

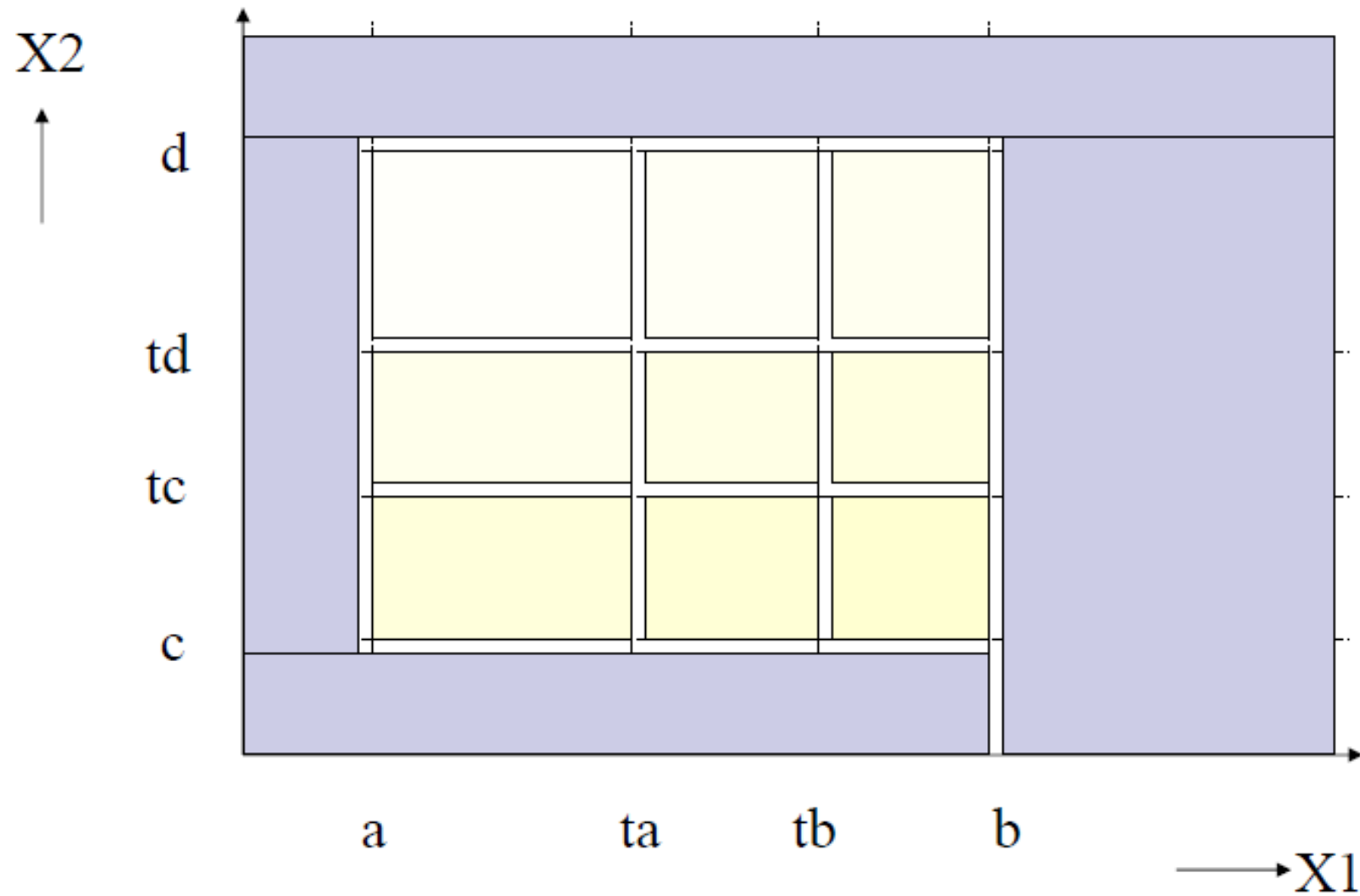
- Extended equivalence classes for x_1

$\{values < a\}, \{a, a+1, \dots, ta\}, \{ta+1, ta+2, \dots, tb\}, \{tb+1, tb+2, \dots, b\}, \{values > b\}$

- Extended equivalence classes for x_2

$\{values < c\}, \{c, c+1, \dots, tc\}, \{tc+1, tc+2, \dots, td\}, \{td+1, td+2, \dots, d\}, \{values > d\}$

Graphical Representation

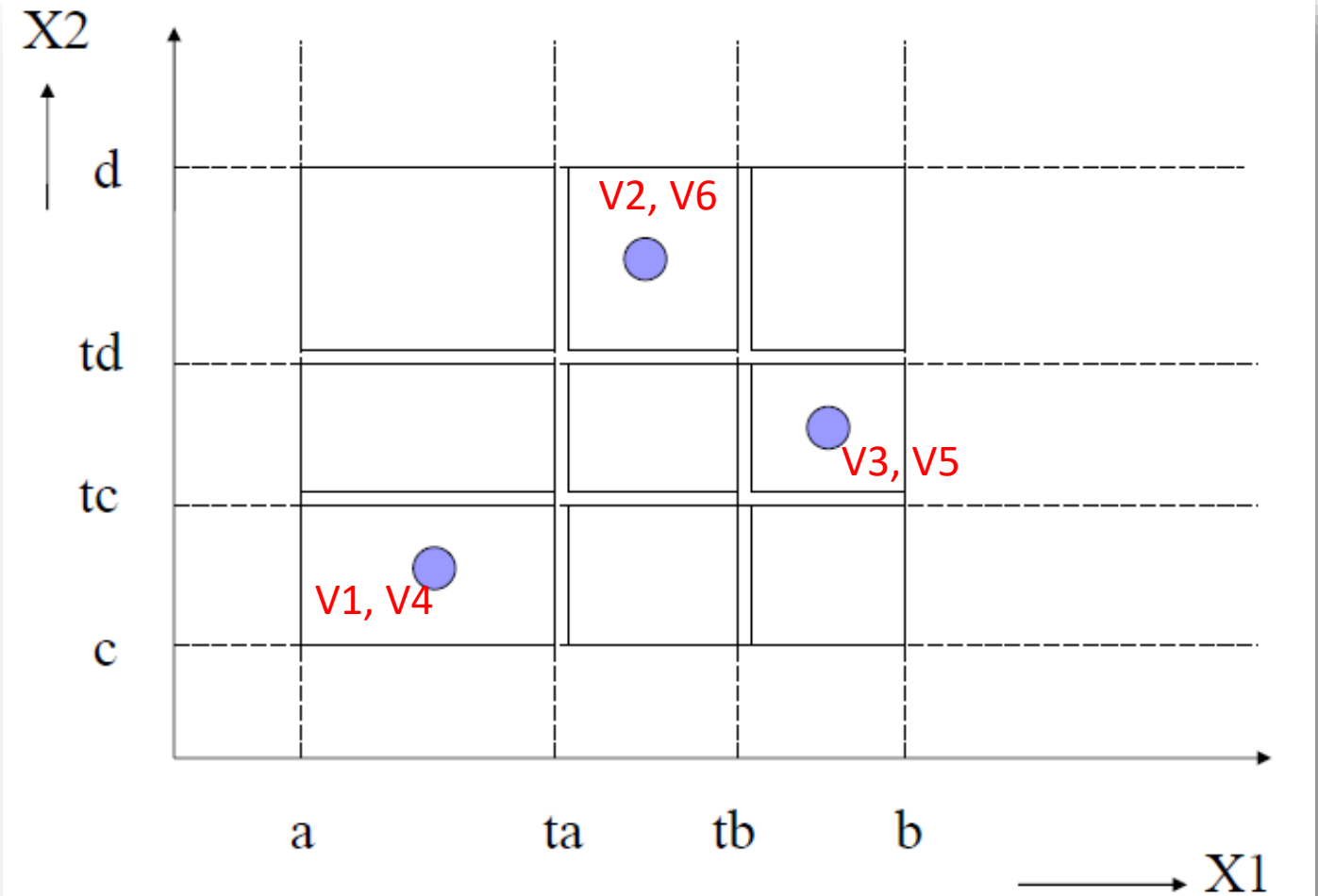


Weak Normal Equivalence Class Testing

- In weak equivalence class testing, the aim is to cover all the equivalence classes at least once, and to do this we don't need to cover all possible combinations.
- The minimum number of test cases is equal to the number of classes in the partition with the largest number of subsets.

Weak Normal Equivalence Class Testing

- Variable x1
 - $V1 = \{a, a+1, \dots, ta\}$
 - $V2 = \{ta+1, ta+2, \dots, tb\}$
 - $V3 = \{tb+1, tb+2, \dots, b\}$
- Variable x2
 - $V4 = \{c, c+1, \dots, tc\}$
 - $V5 = \{tc+1, tc+2, \dots, td\}$
 - $V6 = \{td+1, td+2, \dots, d\}$

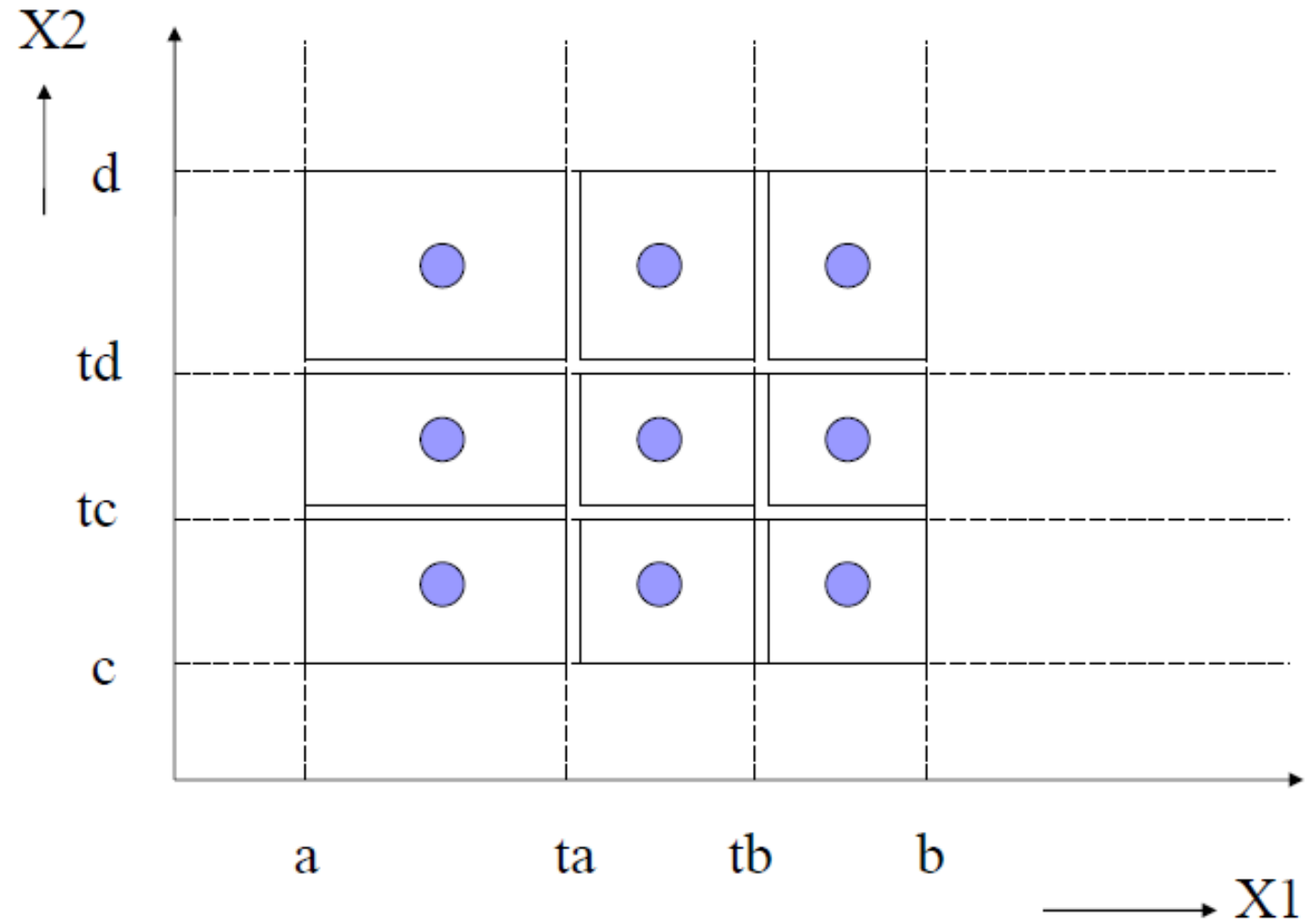


Strong Normal Equivalence Class Testing

- Strong equivalence class testing is based on the Cartesian Product of the partition subsets.
- From the previous example, this would generate:
 - $3 * 3 = 9$ test cases
- Generates more test cases which test for any interaction between the representative values from each of the subsets.

Example

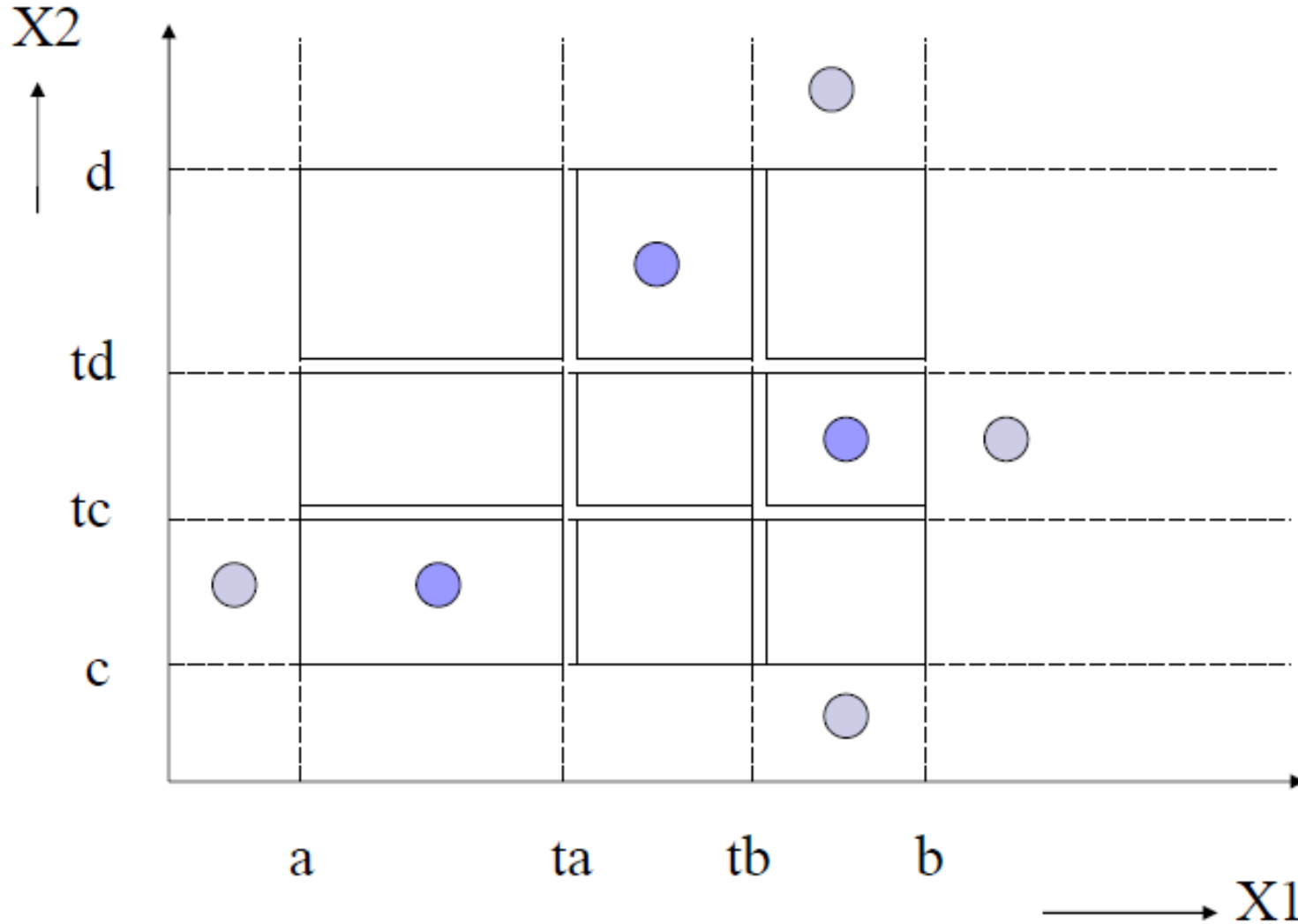
- Variable x1
 - $V1 = \{a, a+1, \dots, ta\}$
 - $V2 = \{ta+1, ta+2, \dots, tb\}$
 - $V3 = \{tb+1, tb+2, \dots, b\}$
- Variable x2
 - $V4 = \{c, c+1, \dots, tc\}$
 - $V5 = \{tc+1, tc+2, \dots, td\}$
 - $V6 = \{td+1, td+2, \dots, d\}$
- Test cases
 - $\langle V1, V4 \rangle, \langle V1, V5 \rangle, \langle V1, V6 \rangle$
 - $\langle V2, V4 \rangle, \langle V2, V5 \rangle, \langle V2, V6 \rangle$
 - $\langle V3, V4 \rangle, \langle V3, V5 \rangle, \langle V3, V6 \rangle$



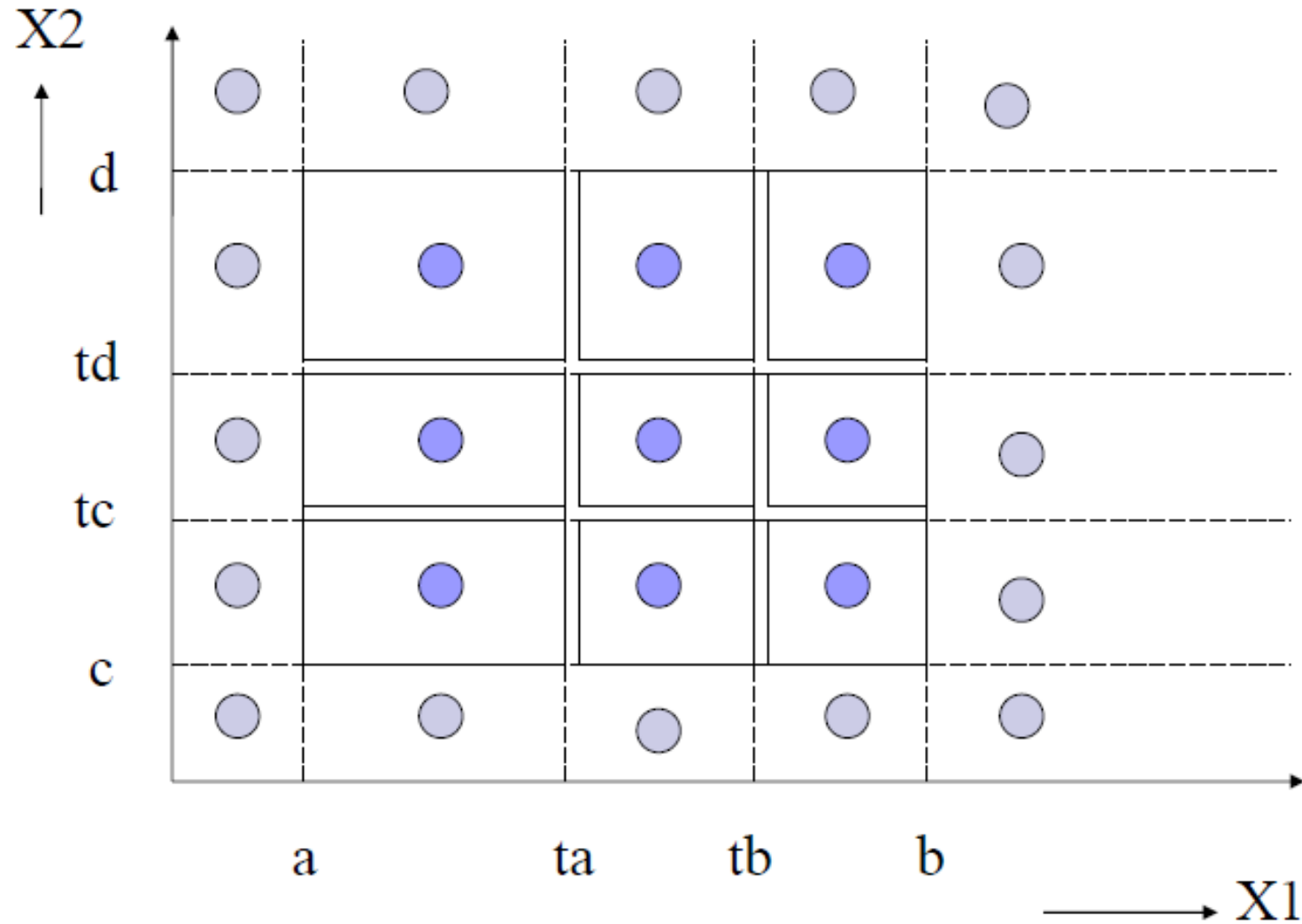
Weak and Strong Robust Equivalence Class Testing

- Weak Robust Equivalence Class Testing
 - Similar to Weak Normal, except additional test cases are chosen from equivalence classes outside the range
 - One test case to cover out of range values for each variable
- Strong Robust Equivalence Class Testing
 - Similar to Strong Normal, except additional equivalence classes are added to cover out of range values
 - Test cases once again derived from the Cartesian product of all equivalence classes

Weak Robust Equivalence class Testing



Strong Robust Equivalence class Testing



Example

- Consider a basic system that accepts two integer inputs, X and Y, and performs a division operation X/Y .
- Here are some equivalence classes for both X and Y:
 - Variable X: {values ≤ -1 }, 0, {values ≥ 1 }
 - Variable Y (denominator): {values ≤ -1 }, 0 (invalid), {values ≥ 1 }



Example

- Consider a basic system that accepts two integer inputs, X and Y, and performs a division operation X/Y .
- Here are some equivalence classes for both X and Y:
 - Variable X: {values ≤ -1 }, 0, {values ≥ 1 }
 - Variable Y (denominator): {values ≤ -1 }, 0 (invalid), {values ≥ 1 }
- Weak Normal Equivalence Class test cases
 - $(X, Y) = (-3, -5), (0, 8), (12, 2)$

Output Equivalence Class

- Equivalence class partitioning can be applied also to the output domain of the software under test.
- Represent the different responses or behaviors that the software may exhibit based on its input.
- Verify whether the software behaves as expected for each class of outputs, which can be more efficient than testing every individual output.

Output Equivalence Class Example

- Consider an example of a simple car rental system where the desired output is the rental cost.
- The system calculates the cost based on the type of car, rental duration (in days), and whether the customer has a membership for a discount.
- Here are the rules:
 1. Economy cars cost \$20 per day, SUVs \$50 per day, and Luxury cars are \$100 per day.
 2. Members get a 10% discount on the total cost.

Output Equivalence Class Example

- The output equivalence classes here would be:
 - 1.Economy car cost for non-members: \$20/day
 - 2.Economy car cost for members: \$20/day with a 10% discount
 - 3.SUV cost for non-members: \$50/day
 - 4.SUV cost for members: \$50/day with a 10% discount
 - 5.Luxury car cost for non-members: \$100/day
 - 6.Luxury car cost for members: \$100/day with a 10% discount

Output Equivalence Class Example

- Weak Normal Equivalence Class Testing, we would choose one representative from each equivalence class and test.
 1. Renting an Economy car for 3 days by a non-member.
 2. Renting an Economy car for 2 days by a member.
 3. Renting an SUV for 4 days by a non-member.
 4. Renting an SUV for 7 days by a member.
 5. Renting a Luxury car for 1 day by a non-member.
 6. Renting a Luxury car for 5 days by a member.

Output Equivalence Class Example

- Strong Normal Equivalence Class Testing, you would now choose one representative from each equivalence class and test for all possible combinations. Here are some representative test cases:
 - 1.Economy car rented by a non-member for 1 day.
 - 2.Economy car rented by a non-member for 2 days.
 - 3.Economy car rented by a member for 1 day.
 - 4.Economy car rented by a member for 2 days.
 - 5.SUV rented by a non-member for 1 day.
 - 6.SUV rented by a non-member for 2 days.
 - 7.SUV rented by a member for 1 day.
 - 8.SUV rented by a member for 2 days.
 - 9.Luxury car rented by a non-member for 1 day.
 - 10.Luxury car rented by a non-member for 2 days.
 - 11.Luxury car rented by a member for 1 day.
 - 12.Luxury car rented by a member for 2 days.

Output Equivalence Class Example

- Robust Equivalence Class Testing involves testing with both valid and 'just outside' invalid inputs.
- In our car rental example, the valid inputs are:
 - 1.Type of car: Economy, SUV, or Luxury.
 - 2.Duration of rental: A positive integer for rental days.
 - 3.Membership status: Whether the customer is a member or not.
- And the invalid inputs could be:
 - 1.Type of car: Any type other than Economy, SUV, or Luxury (e.g., compact, pickup).
 - 2.Duration of rental: Non-positive integers (e.g., 0 or negative numbers).
 - 3.Membership status: Any status other than member or non-member (e.g., pending).

Effective coverage of test cases

- It is preferable to apply equivalence class testing for both input domain and output domain
 - In practice, many people apply the technique only for input domain
- It is preferable to include “Robust equivalence classes”
- It is preferable to combine equivalence class testing with boundary value analysis and apply worst-case scenario (or multiple-fault assumption – more than one variable may be at fault)

Scenario for Equivalence Class Testing

1. Large Input Domain: When the system under test has a large number of inputs, it's impractical to test them all. Equivalence class testing allows you to divide the input into groups (equivalence classes) and test a representative input from each group, greatly reducing the number of test cases.
2. Boundary Conditions: Equivalence class testing is useful for identifying boundary conditions where defects often occur. When you divide the inputs into valid and invalid classes, the boundaries between these classes often reveal errors.
3. Reducing Redundancy: If you believe that testing one value from an equivalence class is equivalent to testing all other values in that class, this method can eliminate redundant or duplicate testing and save considerable time and effort.

Scenario for Equivalence Class Testing

1. Software with Defined Inputs: It works best when the software has clearly defined input domains that can be easily divided into classes. For example, choosing a country from a drop-down list or entering an age in a text box.
2. Quick and Early Bug Discovery: It enables faster testing and helps in identifying high-risk defects at an early stage of testing which improves the quality of the software product.

While equivalence class testing can identify many issues, it doesn't guarantee finding all errors, and should be used as part of a broader testing strategy that includes other techniques as well.

Black-box Testing

- Boundary Value Testing (BVT)
 - Normal boundary value testing
 - Robust boundary value testing
 - Worst-case boundary value testing
 - Robust worst-case boundary value testing
- Equivalence Class Testing
 - Weak Normal Equivalence Class Testing
 - Strong Normal Equivalence Class Testing
 - Weak Robust Equivalence Class Testing
 - Strong Robust Equivalence Class Testing
- **Decision Table Based Testing**

Decision Table

- Decision tables are a precise yet compact way to model complicated logic.
 - Decision tables, like if-then-else and switch-case statements, associate conditions with actions to perform.
 - But, unlike the control structures found in traditional programming languages, decision tables can associate many independent conditions with several actions in an elegant way.
- “http://en.wikipedia.org/wiki/Decision_table”

Decision Table

- Decision tables make it easy to observe that all possible conditions are accounted for.
- Decision tables can be used for Specifying complex program logic.

Conditions/Condition Stubs	Condition Entries
Actions/Action Stubs	Action Entries

Decision Table Structure

<i>Stub</i>	<i>Rule 1</i>	<i>Rule 2</i>	<i>Rules 3, 4</i>	<i>Rule 5</i>	<i>Rule 6</i>	<i>Rules 7, 8</i>
c1	T	T	T	F	F	F
c2	T	T	F	T	T	F
c3	T	F	—	T	F	—
a1	X	X		X		
a2	X				X	
a3		X		X		
a4			X			X

Decision Table Structure

- Each condition corresponds to a variable, relation or predicate
- Possible values for conditions are listed among the condition entries
 - Boolean values (True / False) – Limited Entry Decision Tables
 - Several values – Extended Entry Decision Tables
 - Don't care value
- Each action is a procedure or operation to perform
- The Action entries specify whether the action is to be performed

Decision Table Example

- Consider a function $f(x1, x2)$ where the values of $x1$ and $x2$ are defined to be

$$a \leq x1 \leq b \quad \text{and} \quad c \leq x2 \leq d$$

- When both input values $x1$ and $x2$ are within their ranges (defined in the condition stubs), compute y value (described in the action stubs).
- In all other situations, output an error message

Decision Table Example

$a \leq x1 \leq b$	T	T	F	F
$c \leq x2 \leq d$	T	F	T	F
Compute y	X			
Output error		X	X	X

Decision Table Example

$a \leq x1 \leq b$	T	--	F	F
$c \leq x2 \leq d$	T	F	--	F
Compute y	X			
Output error		X	X	X

‘--’ indicates “don’t care”

Decision Table Example

$x1 < a$	T	F	F	F	F
$x1 = a$	F	T	F	F	F
$a < x1 < b$	F	F	T	F	F
$x1 = b$	F	F	F	T	F
$x1 > b$	F	F	F	F	T
$c \leq x2 \leq d$	--	T	T	T	--
Compute y		X	X	X	
Output error	X				X

Decision Table Example – Extended Entry Decision Table

	Conditions/ Courses of Action	Rules					
		1	2	3	4	5	6
Condition Stubs	Employee type	S	H	S	H	S	H
	Hours worked	<40	<40	40	40	>40	>40
Action Stubs	Pay base salary	X		X		X	
	Calculate hourly wage		X		X		X
	Calculate overtime						X
	Produce Absence Report		X				

Decision Table Development Methodology

1. Determine conditions and values
2. Determine maximum number of rules
3. Determine actions
4. Encode possible rules
5. Encode the appropriate actions for each rule
6. Verify the policy
7. Simplify the rules (reduce the number of columns whenever possible)

Decision Table - Usage

- The use of the decision-table model is applicable when :
 - The specification is given or can be converted to a decision table.
 - The order in which the predicates are evaluated does not affect the interpretation of the rules or resulting action.
 - The order of rule evaluation has no effect on resulting action.
 - Once a rule is satisfied and the action selected, no other rule need be examined.
 - The order of executing actions in a satisfied rule is of no consequence.

Decision Table -Issue

Before using the tables, ensure:

- rules must be complete
 - every combination of predicate truth values plus default cases are explicit in the decision table
- rules must be consistent
 - every combination of predicate truth values results in only one action or set of actions

Inconsistent Decision Table

Conditions	1 -4	5	6	7	8	9
C1	T	F	F	F	F	T
C2	-	T	T	F	F	F
C3	-	T	F	T	F	F
Actions						
a1	X	X	X	-	-	-
a2	-	X	X	X	-	X
a3	X	-	X	X	X	-

Decision Table for Triangle Problem

Conditions											
C1: $a < b+c$?	F	T	T	T	T	T	T	T	T	T	T
C2: $b < a+c$?	-	F	T	T	T	T	T	T	T	T	T
C3: $c < a+b$?	-	-	F	T	T	T	T	T	T	T	T
C4: $a=b$?	-	-	-	T	T	T	T	F	F	F	F
C5: $a=c$?	-	-	-	T	T	F	F	T	T	F	F
C6: $b=c$?	-	-	-	T	F	T	F	T	F	T	F
Actions											
A1: Not a Triangle	X	X	X								
A2: Scalene											X
A3: Isosceles							X		X	X	
A4: Equilateral				X							
A5: Impossible					X	X		X			

Test Cases for Triangle Problem

Case ID	a	b	c	Expected Output
DT1	4	1	2	Not a Triangle
DT2	1	4	2	Not a Triangle
DT3	1	2	4	Not a Triangle
DT4	5	5	5	Equilateral
DT5	?	?	?	Impossible
DT6	?	?	?	Impossible
DT7	2	2	3	Isosceles
DT8	?	?	?	Impossible
DT9	2	3	2	Isosceles
DT10	3	2	2	Isosceles
DT11	3	4	5	Scalene

Observation

- Decision Table testing is most appropriate for programs where
 - Prominent if-then-else logic
 - there are important logical relationships among input variables
 - There are calculations involving subsets of input variables
 - There are cause and effect relationships between input and output
 - There is complex computation logic (high cyclomatic complexity)
- Decision tables do not scale up very well
- Decision tables can be iteratively refined

End